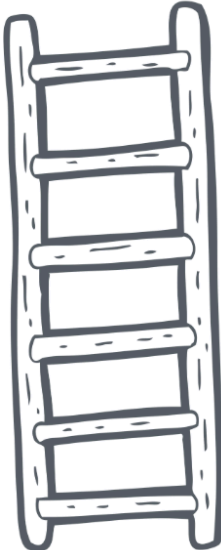


Requirements Analysis Document

WORKSHEETZONE

WORD LADDER

Change only one letter at a time to make a new word for each step.



Fail

Bail

Ball

Bald

Bold

Cold



Jumble Word Puzzle

Unscramble these Jumbles, one letter to each square, to form ordinary words.

NCUDE

--	--	--	--	--

NRILE

--	--	--	--	--

VYRSUE

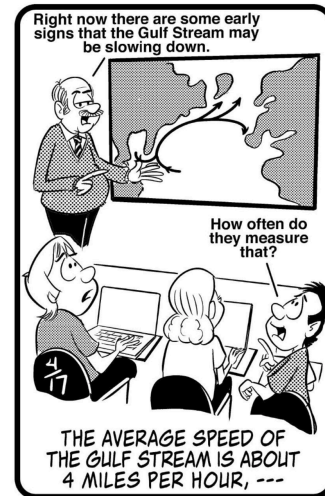
--	--	--	--	--	--

TRWMHA

--	--	--	--	--	--

Arrange the circled letters to form the bonus answer, as suggested by the caption in the cartoon

--	--	--	--	--	--	--	--	--	--



Requirements Analysis Document

FLEX

Project Manager: Daylin Jackson

Team: Luke Hayden

Zamiyah Hill

Dillon McCants

1 Overview	2
1.1 Problem Description	2
1.2 Vision	2
1.3 Business Case	3
1.4 Feature List	3
1.5 Example System	3
2 Proposed System	3
2.1 Functional Requirements	4
2.2 Nonfunctional Requirements	4
2.2.1 Usability	4
2.2.2 Reliability	4

2.2.3 Performance	5
2.2.4 Supportability	5
2.2.5 Implementation	5
2.2.6 Interface	5
2.2.7 Packaging	5
2.2.8 Legal	5
2.3 System Models	5
2.3.1 Scenarios	6
2.3.2 Use Case Model	9
2.3.3 Object Model	11
2.3.4 Dynamic Model	14
2.3.5 User Interface	25
3 Glossary	27

1 Overview

1.1 Problem Description

The client wants us to make two different word games. One of the games is Jumble, and the other is Word Ladder. The client wants us to add varying difficulty levels ranging from easy to hard. The harder the difficulty the more words are added. To make it even harder, we will implement a life system. This means if you get any of the words incorrect, then you are closer to losing the game.

The client also wants to add unique themes and music upon starting a new game. There will also be many different words so that each time the client, or whoever, plays the game, they will be met with a different experience!

1.2 Vision

As the client, I would love a word game with varying difficulty that I can play during my free time. I would love for the game to be playable by anyone, even children, to help them learn. I would like a general friendly environment with randomized themes such as:

- Christmas
- Halloween
- Thanksgiving
- Autumn
- Winter
- Spring

- Summer

And many more, all of which are accompanied by lovely music, and illustrations.

1.3 Business Case

In a business, this project could easily be used as a way to pass time, or it could even be used as a learning tool for children in elementary school. We believe this would help children learn much more efficiently due to the themes, music, and creative thought put into the project. The project should be able to keep the child's attention meaning they can learn new words!

1.4 Feature List

This Project Features:

- Two different types of engaging Word Games
- Varying difficulties from easy to hard
 - A failure system that tallies how many incorrect choices have been made before ultimately ending the game.
- Lovely illustrated themes to keep the player engaged
- Accompanied music to match the generated themes
- Easy to use
- Learnable Material

1.5 Example System

- Websites
 - [Wordle](#)
 - [Vocabulary A-Z](#)
- Newspaper games
- Physical Word game Books
 - Sight Word Trees
 - Word Jumble! : Mumble Jumble Word Scramble Puzzle
- Mobile apps
 - Wordscapes
 - Word Scramble

2 Proposed System

The system we are creating is a Java-based word game application. It will consist of two games, Jumble and Word Ladder. The goal of the system is to offer the users an interactive way to practice their vocabulary, spelling, and even problem solving skills.

The sections below this will describe the functional, nonfunctional requirements, and system models that will create an understanding of the features that the application needs to satisfy our client.

2.1 Functional Requirements

- Main Menu - gives the players a menu that allows them to choose what game they want
- Progression tracker - track users info or progression to see what they've done
- Hint System - allow user to redeem hints if stuck
- Word Ladder - Word Ladder game for the user to play, conventional rules
- Word Ladder Rules - make sure user follows the rules of Word Ladder
- Jumble - have jumble game for user to play
- Jumble Rules - make sure jumble rules are followed correctly
- Multiple Puzzles - have multiple puzzles that can be played with different words
- Scores/Results - allow user to see their progression or completions and scores
- Invalid Input - have cases for invalid inputs from users
- Replay - allow replayability of completed games or puzzles
- Exit Game - allow them to exit game

2.2 Nonfunctional Requirements

Nonfunctional requirements are just as important as functional requirements. Each of these requirements can have a tremendous impact on the project as a whole. Usability is how easy a system is to use. Reliability is how robust the system is. Performance describes the responsiveness of the system. Supportability is how easy the system is to maintain and update. Implementation describes any constraints the system has. The system's interface is what the user uses to interact with it. Packaging deals with the installation of the system. Legal information includes royalties, licensing, and liabilities.

2.2.1 Usability

Both of our word games, Word Ladder and Jumble, should be simple to understand for anyone of any age. Easier difficulties will have common words so that those with a limited English vocabulary can still complete the games. The games will have detailed instructions that are accessible at any time.

2.2.2 Reliability

Our games should be very reliable due to their simple nature. If an error does occur however, the user should either be able to continue their game relatively quickly or not even notice the error at all. Data loss should not be an issue because no data is saved between sessions.

2.2.3 Performance

The user should be able to immediately start a game without waiting for it to load. Users should know if they guessed correctly the second they submit their input. The user should not notice any input latency or sluggish performance.

2.2.4 Supportability

The games should not need to be updated after they are completed, but it should not be hard to update them if necessary. The lead programmer will be in charge of maintaining the games and the overarching system. The games are designed to run in Java, and there are no plans to port them to other programming languages.

2.2.5 Implementation

Our games will run on any computer capable of running Java executables. All updates to code will be done through commits on GitHub. All downloadable files will be hosted on GitHub also.

2.2.6 Interface

The games do not interact with other systems. They are self-contained and run on their own. Data is not imported into or exported from the games. The games do not need to adhere to any standards.

2.2.7 Packaging

The games do not need to be installed; they will both be contained in a .jar file. The user only needs to make sure they can run Java executables beforehand. At least one (the client) installation is foreseen. The games can be downloaded whenever the client wishes to play them.

2.2.8 Legal

Word Ladder was created 150 years ago and never copyrighted, so it has no legal issues associated with recreating it. Jumble, however, is copyrighted. We are free to recreate the rules because we are not making any money off of it. We will be incorporating an MIT License for this project.

2.3 System Models

This section will go over the Model scenarios, what will happen if the user gets their answers correct, or if they get the answer wrong. Use case model, showing the basis of how the system should work. The Object Model, which shows how the systems and

classes should interact with each other. Dynamic Model, showing how everything should function over time. And the User Interface, which displays how the system should visually look.

2.3.1 Scenarios

Fully Dressed Use Case Scenario

Use Case: Play Jumble or Word Ladder

Primary Actor: Player

Goal: The player successfully plays a word game and receives a result.

Preconditions

- The player has opened the word game application
- The game is available to play
- The player has selected either Jumble or Word Ladder.

Main Success Scenario

1. The player selects a word game.
2. The system displays the game instructions and a new puzzle.
3. The player enters an answer.
4. The system checks the player's answer.
5. The system determines whether the answer is correct.
6. If the answer is correct, the system displays a success message and updates the player's puzzle.
7. The system provides the player with the next next puzzle.
8. The player continues playing or chooses to end the game.

Alternative/Exception Flows

1. If the answer is incorrect, the system informs the player that the answer is incorrect.
2. The player can try again if attempts are available.
3. If the player runs out of attempts, the system displays the correct answer and moves to the next puzzle.
4. If the player chooses to quit, the system ends the game and displays the menu screen.

Postconditions

- The game session is completed.
- The player's score is displayed.
- The system is ready for the player to start another game.

Causal Use Case Scenario - Start a Game

Use case: Start Word Game

Primary Actor: Player

Scenario

1. The player application opens the application.
2. The system displays the available games.
3. The player selects Jumble or Word Ladder.
4. The system displays the selected game's instructions/menu.
5. The system generates and displays a puzzle.
6. The player begins playing.

Extensions

- If the player does not select a game, the system continues displaying the game-selection screen.

Casual Use Case Scenario - Submit an Answer

Primary Actor: Player

Scenario

1. The player enters an answer for the current puzzle.
2. The player submits the answer.
3. The system checks the answer.
4. The system informs the player whether the answer is correct or incorrect.
5. If correct, the system updates the player's score.
6. The system allows the player to continue to the next puzzle.

Extensions

- If the answer is incorrect, the system allows the player to try again or continue to the next puzzle, depending on the attempts given.

Brief Use Case Scenario - View Instructions

Use case: View Game Instruction

Primary Actor: Player

Description: The player selects which game they want to play, and the system displays the information explaining how to play the game next.

Brief Use Case Scenario - View Score

Use Case: View Game Score

Primary Actor: Player

Description: The player requests their current score, and the system displays the player's score for the current game.

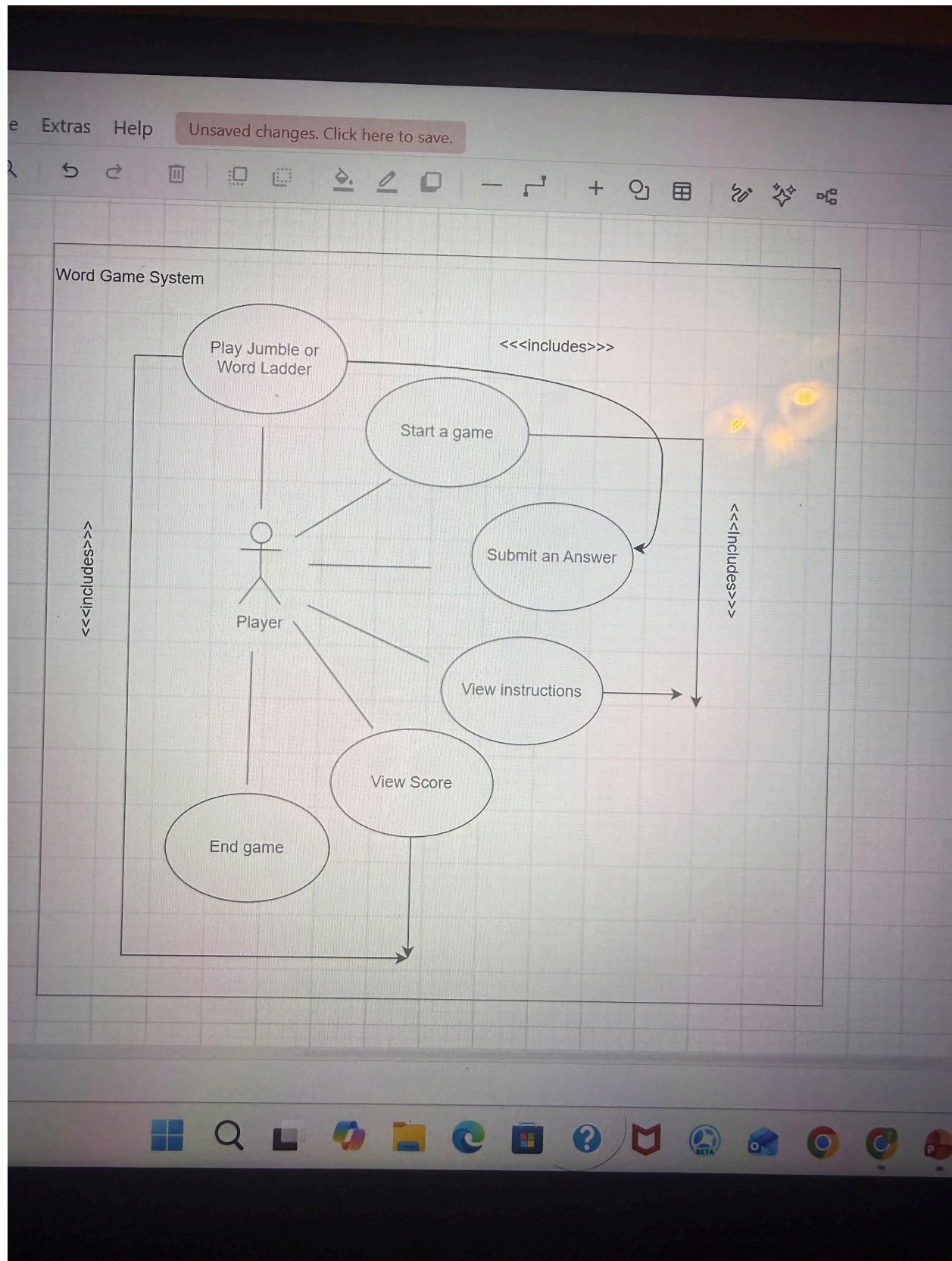
Brief Use Case Scenario - End Game

Use Case: End game

Primary Actor: Player

Description: The player chooses to end the current game, and the system ends the game session and displays the player's final score.

2.3.2 Use Case Model



Actor: Player

System Boundary: Word Game System

Use Cases Inside the Boundary:

1. Player Jumble or Word Ladder
2. Start a Game
3. Submits an answer
4. View Instructions
5. View Score
6. End Game

Play Jumble or Word ladder INCLUDES Submit an answer:

Because submitting an answer is a required part of playing either game.

Playing Jumble or Word Ladder INCLUDES View Score:

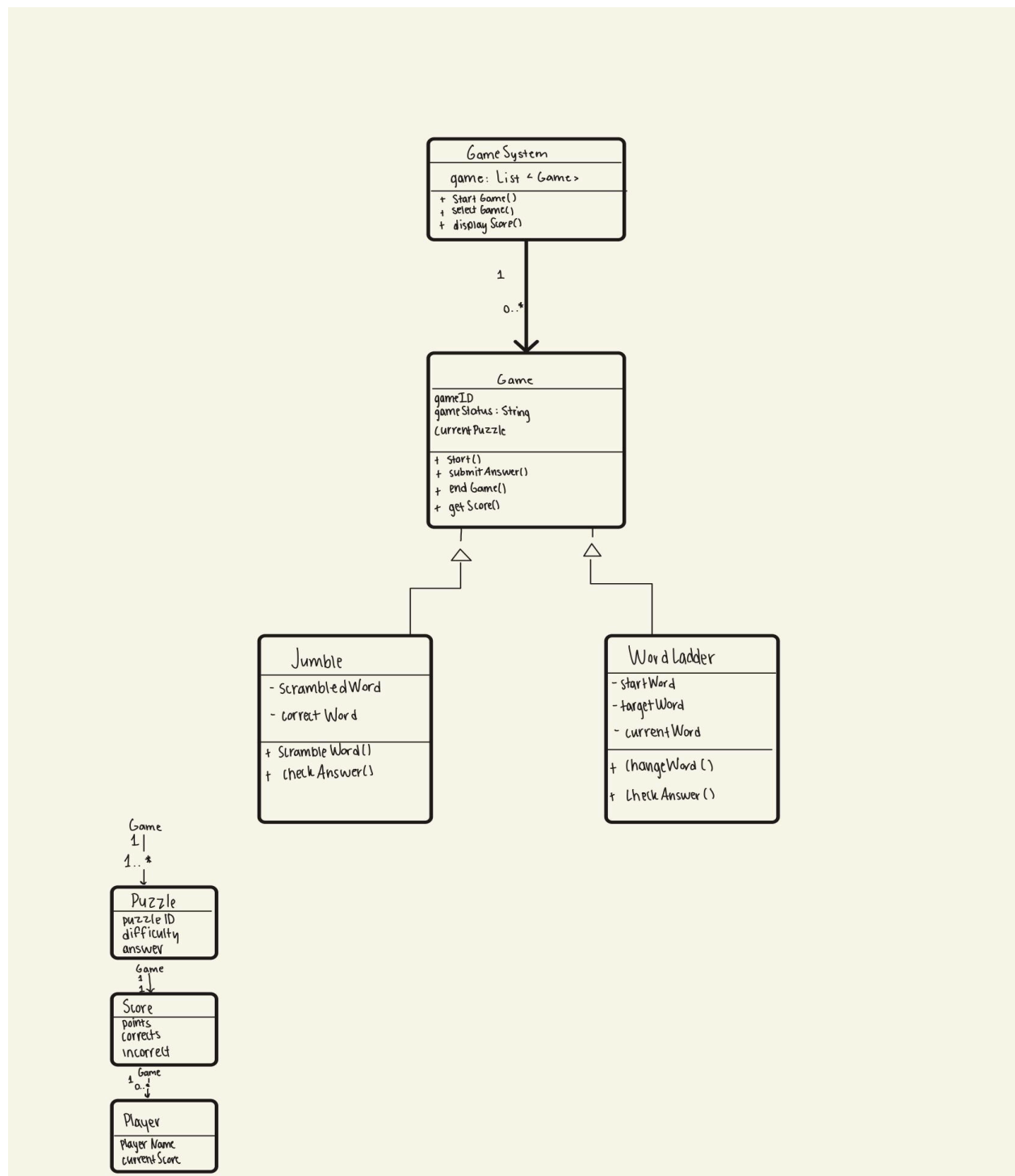
If the score is automatically shown/ updated as part of playing

Start A Game INCLUDES View Instructions:

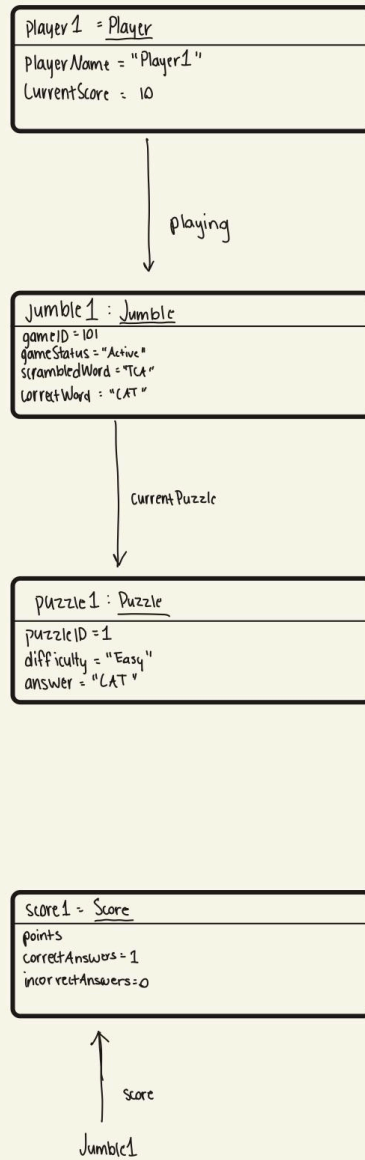
If the instructions are always displayed before the game begins.

2.3.3 Object Model

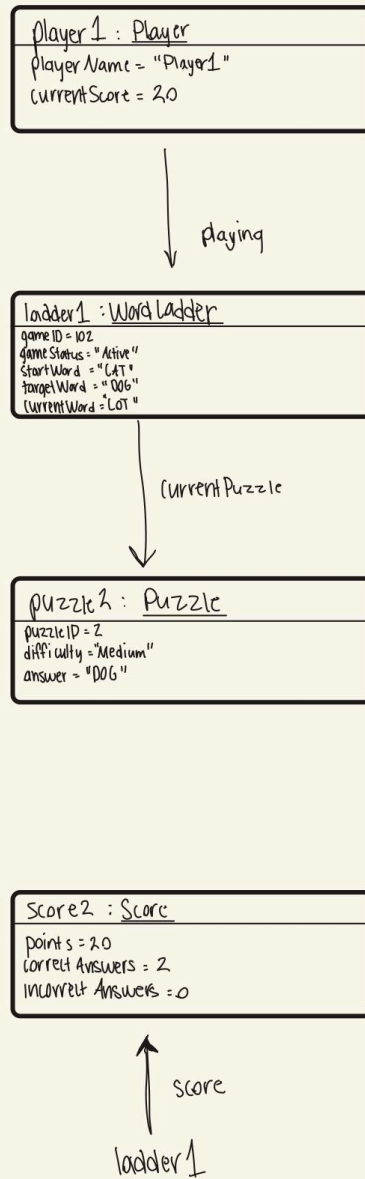
UML Class Diagram



UML Object Diagram - Jumble Game



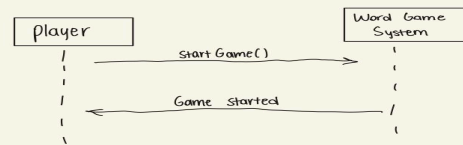
UML Object Diagram - Word Ladder Game



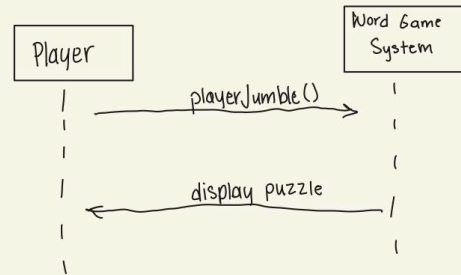
2.3.4 Dynamic Model

Sequence Diagram 1 - Start a Game

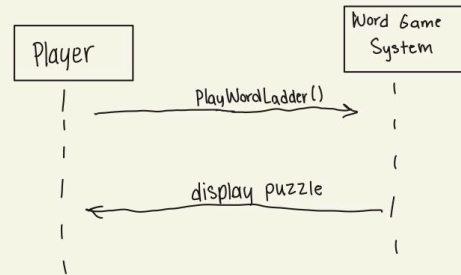
Actor: Player



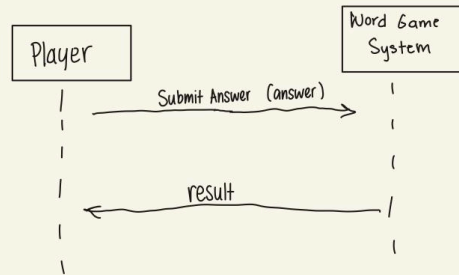
Sequence Diagram 2 - Play Jumble



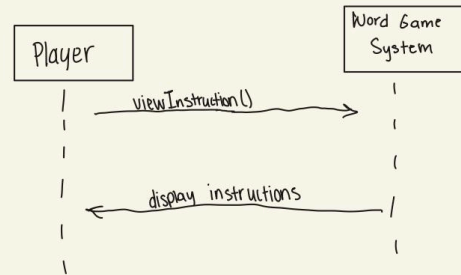
Sequence Diagram 3 - Play Word Ladder



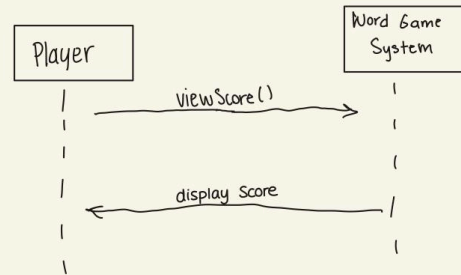
Sequence Diagram 4 - Submit Answer



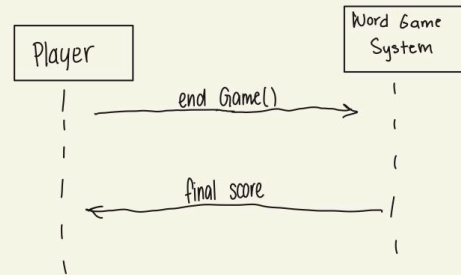
Sequence Diagram 5 - View Instructions



Sequence Diagram 6 - View Score



Sequence Diagram 7 - End Game



Operation Contracts

Operation Contract 1 - Start Game

Operation: startGame(gameType)

Cross References: Start a Game

Preconditions:

- The player is on the main menu
- The player has selected either Jumble or Word Ladder

Postconditions:

- A new game session is created.
- The selected game type is stored.
- The game status is set to Active.
- A puzzle is selected and displayed.
- The player's score is initialized.

Operation Contract 2 - Play Jumble

Operation: playJumble()

Cross References: Play Jumble

Preconditions:

- A player has selected Jumble
- The game is active.

Postconditions:

- A Jumble puzzle is selected.
- The word is scrambled.
- The scrambled word is displayed to the player.
- The game waits for the player's answer.

Operation Contract 3 - Play Word Ladder

Operation: playWordLadder()

Cross References: Play Word Ladder

Preconditions:

- A player has selected Word Ladder
- The game is active.

Postconditions:

- A Word Ladder puzzle is selected.
- The starting word and target are displayed.

- The current word is set to the starting word.
- The game waits for the player's next word.

Operation Contract 4 - Submit an answer

Operation: submitAnswer(answer)

Cross References: Submit an Answer

Preconditions:

- A game is currently active.
- A puzzle is currently displayed.
- The player has entered an answer.

Postconditions:

- The system compares the submitted answer with the expected answer.
- The system determines whether the answer is correct.
- If the answer is correct, the player's score is increased.
- If the answer is incorrect, the score remains unchanged.
- The result is displayed to the player.
- If the puzzle is completed, the system provides the next puzzle.

Alternative outcome:

If the answer is incorrect:

- The system displays an incorrect answer message.
- The player is allowed to try again if attempts are available.

Operation Contract 5 - View Instructions

Operation: viewInstruction()

Cross References: View Instructions

Preconditions:

- The word game application is open

Postconditions:

- The system displays instructions for Jumble and Word Ladder.
- The player can return to the main menu or select a game.

Operation Contract 6 - View Score

Operation: viewScore()

Cross References: View Score

Preconditions:

- A game session exists.

Postconditions:

- The system retrieves the player's current score.
- The current score is displayed to the player.
- The player can return to the active game or main menu.

Operation Contracts 7 - End Game

Operation: endGame()

Cross References: End Game

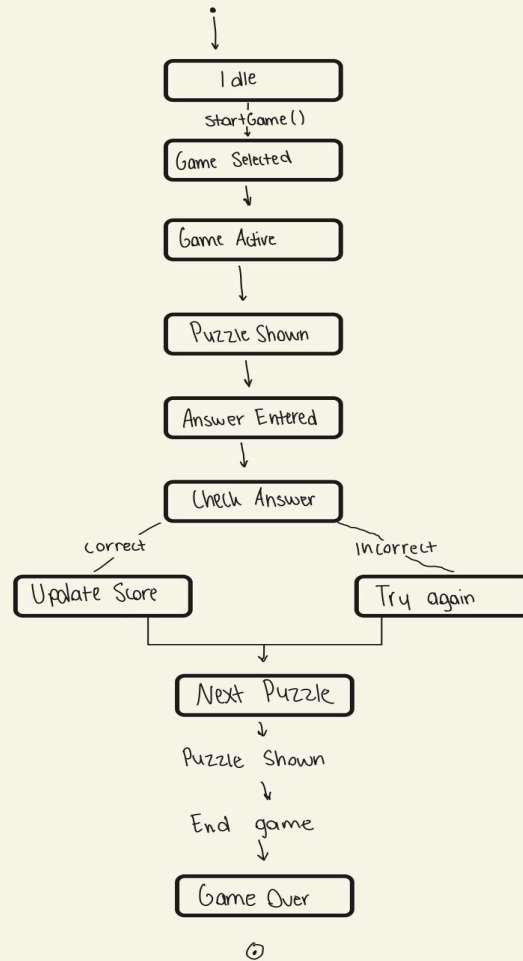
Preconditions:

- A game session is currently active.

Postconditions:

- The current game session has ended.
- The player's final score is calculated.
- The game status is changed to completed.
- The player is returned to the main menu.

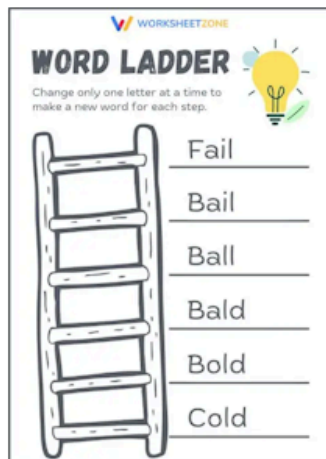
UML State Diagram



2.3.5 User Interface

Word Games

Word Ladder



Play

Jumble

Jumble Word Puzzle

Unscramble these Jumbles, one letter to each square, to form ordinary words.

NCUDE

NRILE

VYRSUE

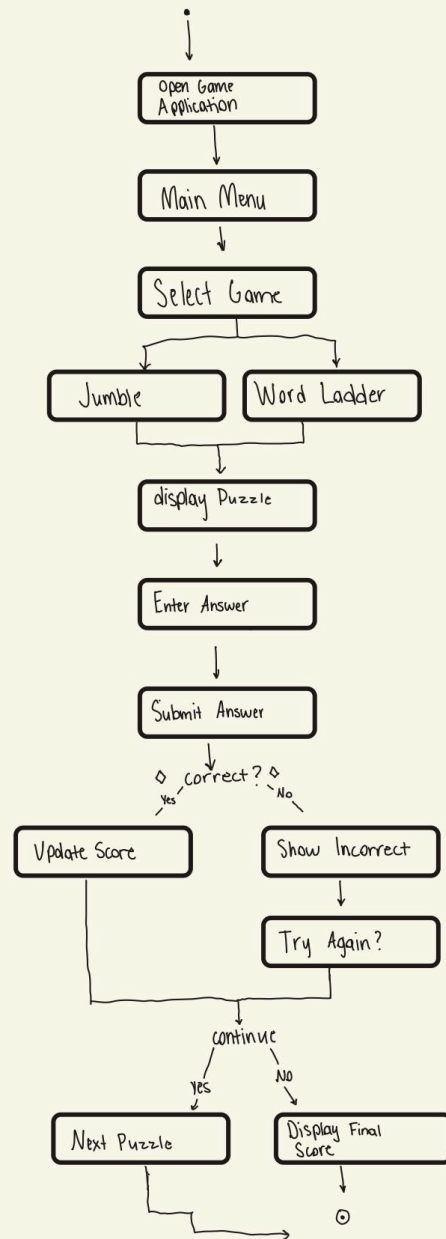
TRWMHA

Arrange the circled letters to form the bonus answer, as suggested by the caption in the cartoon.



Play

UML Activity Diagram



In the diagram above, it shows the overall activity flow of the Word Game System. The player begins at the main menu and selects either Jumble or Word Ladder. The system displays the appropriate puzzle, accepts the player's answer, checks the answer, updates the score when appropriate, and allows the player to continue to another puzzle or end the game.

3 Glossary

MIT License - Massachusetts Institute of Technology License

UML - Unified Modeling Language